

Universidad Tecnológica de Bolívar

UNIVERSIDAD TECNOLOGICA DE BOLIVAR

TALLER DEL SEGUNDO CORTE: DOCKER

Parte 1. Informe de investigación

Asignatura: Arquitectura de Software

Estudiante: Guillermo Andrés Rodríguez Munive

Profesor: Luis Giovanny Carreno Ortiz

16 de octubre de 2026

Taller del segundo corte: Docker

Parte 1. Informe de investigación
Asignatura: Arquitectura de Software
Estudiante: Guillermo Andrés Rodríguez Munive
Fecha: 16 de octubre de 2026

Introducción

Al desarrollar un programa no basta con que el código esté bien escrito. También es importante que funcione en distintos computadores y que las herramientas necesarias para ejecutarlo estén configuradas correctamente. Docker ayuda a resolver esta dificultad porque permite empaquetar aplicaciones y sus dependencias dentro de contenedores. En este informe se explican sus conceptos básicos, componentes, funciones y comandos principales. También se analiza su utilidad para organizar y desplegar sistemas que están compuestos por varios servicios.

Desarrollo

1. ¿Qué es Docker?

Docker es una plataforma que permite crear, distribuir y ejecutar aplicaciones utilizando **contenedores**. Un contenedor es un entorno aislado donde se ejecuta una aplicación con sus archivos, bibliotecas y configuraciones necesarias. Gracias a esto, no es necesario instalar manualmente todas esas dependencias en cada computador.

Los contenedores comparten el **núcleo (kernel)** del sistema operativo del host donde se ejecuta el motor de Docker. Por eso no necesitan iniciar un sistema operativo completo por separado. En Windows, Docker Desktop normalmente utiliza un entorno Linux mediante WSL 2 para ejecutar contenedores Linux; en ese caso comparten el kernel de ese entorno Linux, no directamente el de Windows.

La diferencia frente a una **máquina virtual** es que esta incluye su propio sistema operativo invitado y requiere más recursos para ejecutarlo. Un contenedor, en cambio, utiliza el kernel del host y suele iniciar más rápido y consumir menos memoria. Esto no significa que sea idéntico a una máquina virtual ni que ofrezca exactamente el mismo nivel de aislamiento.

2. ¿Cuáles son sus componentes?

Componente	Explicación
CLI (docker)	Es el cliente que usamos en la terminal para enviar instrucciones, por ejemplo iniciar o detener contenedores.
Daemon (dockerd)	Es el proceso que recibe las solicitudes del cliente y administra las imágenes, los contenedores, redes y volúmenes.
Imagen	Es una plantilla de solo lectura que contiene los archivos y la configuración necesarios para crear un contenedor.
Contenedor	Es una instancia ejecutable de una imagen. Se puede iniciar, detener o eliminar sin cambiar la imagen original.
Dockerfile	Es un archivo con las instrucciones para construir una imagen personalizada.

Componente	Explicación
Registro	Es el lugar donde se almacenan y distribuyen imágenes. Un ejemplo es Docker Hub.

La relación entre estos componentes se puede resumir así:

```
Dockerfile --(docker build)--> Imagen --(docker run)--> Contenedor
                                 ^
                          docker pull / docker push
                                 |
                           Registro (Docker Hub)

CLI docker --> Daemon dockerd --> administra imágenes y contenedores
```

Por ejemplo, en biblioteca-api el Dockerfile permite construir la imagen de la API escrita en Go. Docker Compose utiliza esa imagen para crear el contenedor de la API y obtiene del registro la imagen mysql:8.0 para la base de datos.

3. Principales funcionalidades

Funcionalidad	¿Para qué sirve?	Ejemplo concreto
Portabilidad	Facilita trasladar una aplicación entre equipos compatibles sin volver a configurar sus dependencias.	Llevar una API que funciona en un portátil a un servidor con Docker.
Aislamiento	Permite ejecutar programas con sus propias dependencias y procesos, evitando muchos conflictos.	Mantener MySQL 8 en un contenedor sin instalarlo directamente en el computador.
Entornos reproducibles	Ayuda a que todos los integrantes del equipo trabajen con configuraciones similares.	Varios estudiantes levantan la misma API y base de datos con docker compose up.
Versionado de imágenes	Permite identificar versiones mediante etiquetas y controlar las actualizaciones.	Usar mysql:8.0 en lugar de depender de la versión instalada en cada equipo.
Despliegue y automatización	Hace posible construir y probar imágenes dentro de procesos de integración y entrega continua (CI/CD).	Un flujo de GitHub Actions crea una imagen después de aprobar las pruebas.
Escalabilidad	Facilita crear otras instancias de un servicio cuando la arquitectura lo permite.	Levantar más réplicas de una API detrás de un balanceador de carga.

4. Principales comandos

a) Tabla de comandos

Comando	¿Qué hace?	Ejemplo
docker pull	Descarga una imagen desde un registro.	docker pull mysql:8.0
docker run	Crea e inicia un contenedor a partir de una imagen.	docker run --name prueba nginx:alpine
docker ps	Muestra los contenedores en ejecución.	docker ps
docker ps -a	Muestra todos los	docker ps -a

Comando	¿Qué hace?	Ejemplo
	contenedores, incluso detenidos.	
docker logs	Consulta los mensajes producidos por un contenedor.	docker logs biblioteca_api
docker exec	Ejecuta un comando dentro de un contenedor activo.	docker exec -it biblioteca_api sh
docker stop / start	Detiene o vuelve a iniciar un contenedor existente.	docker stop biblioteca_api / docker start biblioteca_api
docker rm	Elimina un contenedor detenido.	docker rm prueba
docker images	Lista las imágenes guardadas localmente.	docker images
docker rmi	Elimina una imagen local que no esté siendo usada.	docker rmi nginx:alpine
docker build	Construye una imagen a partir de un Dockerfile.	docker build -t biblioteca-api:local .
docker compose up	Crea e inicia los servicios descritos en el archivo Compose.	docker compose up --build

b) Explicación de tres comandos

docker run: lo usaría cuando quiero iniciar un contenedor de manera individual. Por ejemplo:

```
docker run -d --name mysql-prueba -e MYSQL_ROOT_PASSWORD=clave_prueba \
  -p 3307:3306 -v datos_prueba:/var/lib/mysql mysql:8.0
```

El parámetro -d lo ejecuta en segundo plano; --name le asigna un nombre; -e define una variable de entorno; -p conecta el puerto 3307 del computador con el 3306 del contenedor; y -v monta un volumen para conservar los datos. La contraseña del ejemplo solo es para una prueba local, no para producción.

docker exec: permite ingresar o ejecutar instrucciones en un contenedor que ya está funcionando. Con docker exec -it biblioteca_api sh, -i mantiene la entrada interactiva y -t habilita una terminal. Lo usaría para revisar archivos o diagnosticar un problema dentro de la API.

docker compose up: permite iniciar varios servicios relacionados. En docker compose up --build, --build indica que se construya la imagen de los servicios que tienen una sección build antes de iniciar. También podría ejecutar docker compose up -d --build si necesito dejar API y MySQL en segundo plano. Lo usaría para levantar el sistema completo de biblioteca, no cada servicio por separado.

5. ¿Qué problema resuelve Docker?

Uno de los inconvenientes habituales al programar es la frase **«en mi máquina funciona»**. Esto ocurre cuando una aplicación se ejecuta bien en el equipo del desarrollador, pero falla en otro porque tiene versiones diferentes de los programas, le faltan dependencias o utiliza una configuración distinta. Las instalaciones manuales aumentan la posibilidad de errores.

Por ejemplo, antes de usar Docker, un estudiante podía instalar una versión de MySQL diferente de la que necesitaba una API, configurar mal las credenciales y perder tiempo intentando conectarla. Con Docker, el proyecto puede especificar qué imagen de MySQL utiliza y qué configuración necesita la API. Así, quien recibe el proyecto ejecuta la misma definición de

servicios y reduce las diferencias entre entornos. Aun así, deben configurarse correctamente las variables, puertos y permisos; Docker no elimina todos los problemas posibles.

6. Importancia dentro de la Arquitectura de Software

En una aplicación compuesta por distintos servicios conviene mantener responsabilidades separadas. Por ejemplo, una **API** atiende solicitudes, una **base de datos** almacena información y un **broker de eventos** distribuye mensajes. Si cada uno se ejecuta en su propio contenedor, resulta más sencillo administrarlo, revisar sus registros y actualizarlo sin mezclar sus dependencias con las de los demás.

Docker también ayuda a conservar un entorno similar durante el paso de **desarrollo a pruebas y producción**: se puede construir una imagen y desplegar esa misma versión en las etapas posteriores, cambiando únicamente la configuración específica de cada ambiente. Además, facilita ejecutar múltiples instancias de un servicio y llevarlas a servidores o plataformas de nube, aunque la distribución de tráfico y la coordinación de esas instancias requieren herramientas adicionales.

Esta forma de trabajar se relaciona con los **microservicios**, donde cada servicio cumple una función concreta; con la **arquitectura orientada a servicios (SOA)**, donde varios componentes se comunican mediante interfaces; y con los sistemas **orientados a eventos**, donde pueden intervenir brokers como RabbitMQ. Docker no obliga a usar uno de estos estilos, pero facilita desplegar sus componentes. En el proyecto biblioteca-api, por ejemplo, una API organizada por capas se comunica con MySQL mediante una red interna de Compose.

Conclusión

Considero que Docker es útil porque permite concentrarse en el desarrollo de una aplicación sin tener que configurar desde cero todas sus dependencias en cada equipo. Me parece especialmente práctico cuando un sistema necesita una API y una base de datos que deben trabajar juntas. Lo utilizaría en un proyecto de inventario, separando el servicio que procesa compras y ventas de la base de datos. De esta manera, sería más fácil probar el sistema, compartirlo y trasladarlo a otro computador manteniendo una configuración conocida.

Referencias

1. Docker Inc. (s. f.). *What is Docker?* Documentación oficial. https://docs.docker.com/get-started/docker-overview/
2. Docker Inc. (s. f.). *How Compose works*. Documentación oficial. https://docs.docker.com/compose/intro/compose-application-model/
3. Docker Inc. (s. f.). *Volumes*. Documentación oficial. https://docs.docker.com/engine/storage/volumes/
4. Docker Inc. (s. f.). *What is an image?* Documentación oficial. https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-an-image/
5. carrenoutb. (2026). *Biblioteca API* [Repositorio y guía de ejecución]. GitHub. https://github.com/carrenoutb/biblioteca-api